Exp No: 27

BINARY SEMAPHORE IN SYSTEMC

AIM:

To implement and simulate a binary semaphore in SystemC for task synchronization and mutual exclusion.

PROCEDURE:

1. Initialize the semaphore variable to 1, indicating that the critical section is available.
2. Create two concurrent processes, task1() and task2(), using SC_THREAD.
3. Make each task check the semaphore before entering the critical section.
4. Set the semaphore to 0 while a task is executing and release it by setting it back to 1 after completion.
5. Run the simulation for 10 seconds and observe task execution times to verify mutual exclusion.

PROGRAM:

```
#include <systemc.h>


SC_MODULE(binary_semaphore) {


int sem;

sc_event ev;


void task1() {

while(true) {

wait(1, SC_SEC);


if(sem == 1) {


sem = 0;

cout << "Task1 Entered Critical Section at " << sc_time_stamp() << endl;

wait(2, SC_SEC);
```

```
sem = 1;
ev.notify();
}
else {
wait(ev);
}
}
}

void task2() {
while(true) {
wait(2, SC_SEC);

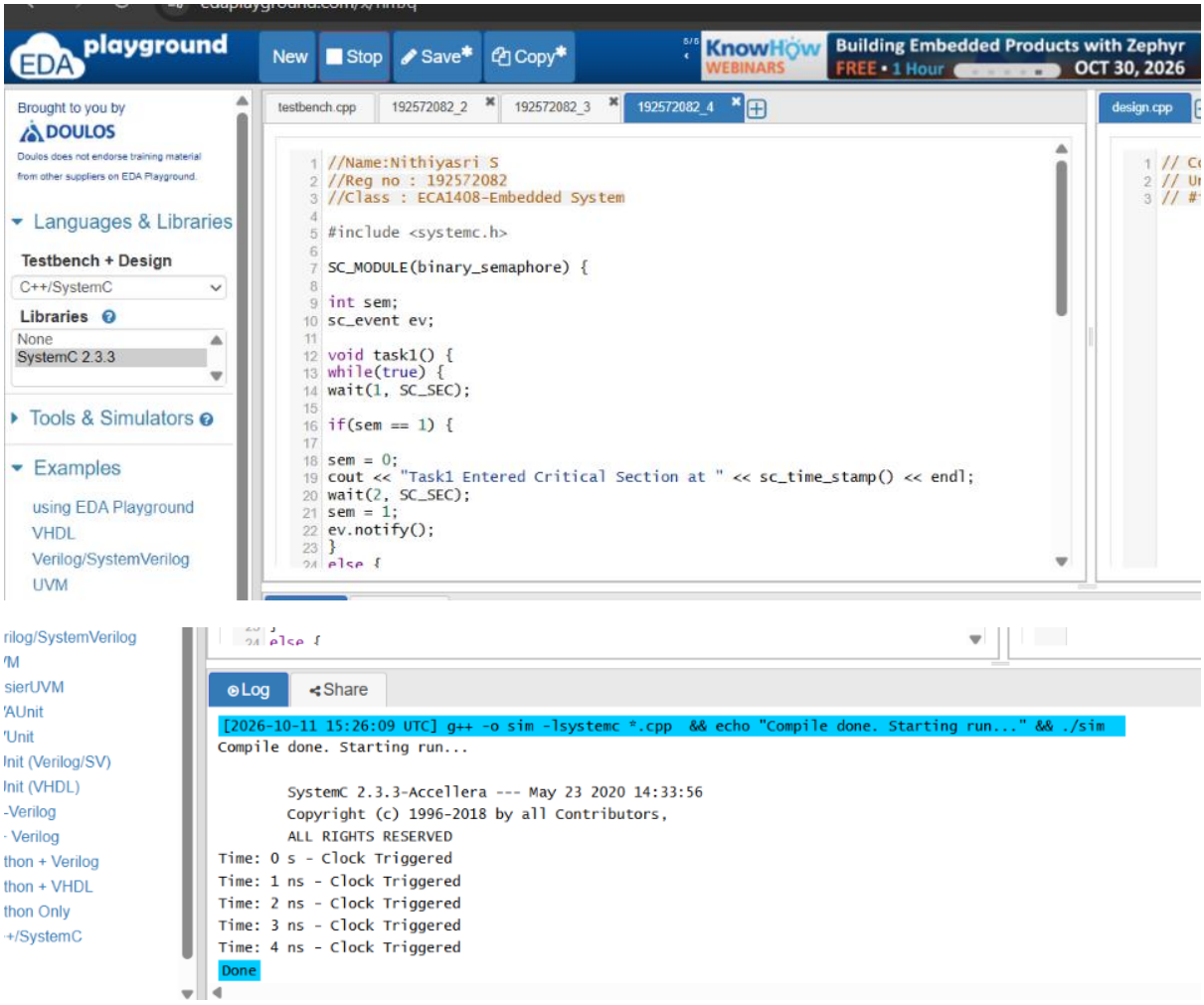
if(sem == 1) {
sem = 0;
cout << "Task2 Entered Critical Section at " << sc_time_stamp() << endl;
wait(1, SC_SEC);
sem = 1;
ev.notify();
}
else {
wait(ev);
}
}
}

SC_CTOR(binary_semaphore) {
sem = 1;
SC_THREAD(task1);
```

```
SC_THREAD(task2);
}

};

int sc_main(int argc, char* argv[]) {
binary_semaphore obj("Binary_Semaphore");
sc_start(10, SC_SEC);
return 0;
}
```



RESULT:

Thus, the binary semaphore was implemented and simulated using SystemC, demonstrating synchronization between concurrent tasks and mutual exclusion in the critical section.